

OpsFlow 功能状态

功能状态 — 已完成/待实现 + Pipeline 格式 + 状态机

2026-06-03

Generated: 2026-06-03 | Source: backend/opsflow/doc/

已完成功能

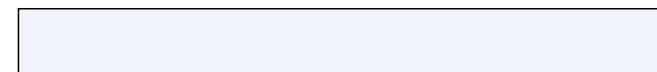
后端核心

功能	模块	说明
数据模型	`models/` 包 (11 模块)	FlowTemplate / TemplateVersion / FlowExecution / OpsLog / SchedulePlan / OpsKnowledge / PluginMeta / NodeExecutionTrace / OpsProject / ProjectMember / TemplateNode / ExecutionNode / ExecutionScheme / TemplateCollect / TemplateCategory / OperationRecord / WebhookConfig / WebhookLog / AutoRetryStrategy / NodeTimeoutConfig / ApiToken / ProjectEnvironmentVariable
API 路由	`urls.py`	12 组 REST ViewSet + 11 个 dashboard + CMDb + API GW + WebSocket
Pipeline 构建	`bamboo_builder.py`	自定义 nodes/edges 格式 → bamboo-engine Pipeline Tree
Pipeline 兼容性验证	`bamboo_validator.py`	网关配对/出入度/环检测/条件引用校验
FlowEngine 迁移	`flow_engine.py`	自定义解释器 → BambooDjangoRuntime + api.run_pipeline()
串行执行	`flow_engine.py`	通过 BambooDjangoRuntime 驱动
ExclusiveGateway	`flow_engine.py`	委托 BambooDjangoRuntime.GatewayMixin
ParallelGateway	`flow_engine.py`	委托 BambooDjangoRuntime.GatewayMixin
ConditionalParallelGateway	`flow_engine.py`	委托 BambooDjangoRuntime
ConvergeGateway	`flow_engine.py`	委托 BambooDjangoRuntime.ConvergeMixin
暂停/继续	`flow_engine.py`	api.pause_pipeline() / api.resume_pipeline()
重试/跳过	`flow_engine.py` + `node_dispatcher.py`	api.retry_node() / api.skip_node() / 强制失败
批量重试/跳过	`execution_node_command.py`	遍历失败节点自动批量操作
取消/终止	`flow_engine.py`	api.revoke_pipeline() + 标记 cancelled
子流程重试	`flow_engine.py`	retry_subprocess 委托执行

审批流程



`execution\_approval.py`



approve/reject/pending_approval 完整审批



Component 注册

`plugin\_service\_adapter.py`





post_set_state 信号



`signals/` (6 模块)



signals.py → signals/ 包拆分，异步追踪节点状态

插件系统

`plugins/` (12 组 51+ 原子)

BasePlugin + 自动发现 + PluginMeta 持久化 + 多版本

安全校验

`safety\_guard.py`

AI 幻觉防御

```
`template_ai.py` + `llm_service.py`
```


AI 生成

`llm\_service.py`

AI 多轮对话

`llm\_service.py`

增量修改已有流程

AI 分析

`llm\_service.py`

自动布局

core/layout/

Sugiyama 引擎 (bk_sops 算法, 5 阶段, 无 LLM 依赖)

RAG 检索

`llm\_service.py`

功能	模块	说明
Ansible Tower 集成	`tower_service.py` + `ansible_trigger.py`	launch/poll/artifacts/events/cancel + Mock 降级
WebSocket	`consumers.py`	节点状态 + tower_job_update 实时推送
调度计划	`models/schedule.py` + `scheduler_service.py`	SchedulePlan 模型 + APScheduler（一次性/CRON）+ 模板快照
调度 API	`schedule_views.py`	CRUD + pause/resume/trigger/history
调度管理前端	`schedule.vue` / `ScheduleManager.vue` / `ScheduleForm.vue` / `ScheduleTable.vue`	调度计划管理 UI
独立调度进程	`start_opsflow_scheduler.py`	APScheduler 独立进程 + Redis 锁防重复
全局变量系统	`template_variable.py` + `variable_resolver.py`	变量 CRUD / 引用计数 / 自动清理 / 变量浏览器
变量提升/解除	`template_variable.py`	节点参数提升为全局变量 + 解除关联
子流程版本追踪	`template_subprocess.py`	版本过期检查 + 批量引用更新
导入/导出模板	`template_export.py`	JSON 包含版本历史 / 导入草稿
执行轨迹双树	`models/execution.py` / `node_dispatcher.py` / `trace_logger.py`	NodeExecutionTrace 模型 + FlowExecution.state_tree + 独立日志文件
节点日志文件	`core/trace_logger.py`	NodeTraceLogger (JSON Lines 格式, 按 execution 分组)
日志清理命令	`clean_node_trace_logs.py`	保留 N 天, 支持 dry-run
数据清理命令	`clean_opsflow_data.py`	清理过期执行/调度/版本数据
种子知识库	`seed_knowledge.py`	24 条 IT 运维知识条目
视图 Mixin 重构	`views/mixins/`	11 个 Mixin 提取, 大文件拆分
signals 包重构	`signals/`	单文件 398 行 → 6 模块包
Dashboard	`dashboard_views/` (3 模块)	11 个端点 (stats/trends/analytics) + 包拆分
插件 API	`plugin_views.py`	只读列表 + 详情 (版本过滤) + 分组树 + 变量类型
错误码体系	`core/error_codes.py`	ErrorCodes 分类 + api_success/api_error 统一返回值
节点操作调度器	`core/node_dispatcher.py`	NodeCommandDispatcher (retry/skip/force_fail/trace 标准化)

轨迹 API



`execution\_trace.py`



GET /traces/ + /trace_log/



变量注册表


```
`core/variable_registry.py` + `variables/common.py`
```



SpliceVariable/LazyVariable 模式 (bk_sops 适配)

子流程调度器

`core/subprocess\_dispatcher.py`



独立子流程 FlowExecution 创建 + 变量映射

项目管理

```
`models/project.py` + `project_views.py`
```


项目隔离

`views/base.py`

ProjectFilteredViewSet 自动按项目角色过滤

数据模型包拆分

`models/` 包

单 models.py → 11 模块包

自动重试

`core/auto\_retry.py`

AutoRetryStrategyCreator + dispatch_auto_retry

节点超时策略

``core/node_timeout_strategy.py``

超时检测 + 强制失败/跳过动作

节点持久化同步

`core/node\_sync.py`

pipeline_tree TemplateNode/ExecutionNode 双向同步

Mako 变量解析

`core/mako\_resolver.py`

Mako 模板变量替换

冲突检测

``core/conflict_checker.py``

多人编辑冲突检测

操作审计日志

`core/audit\_logger.py`

OperationRecord 自动记录

功能	模块	说明
插件生命周期	`core/plugin_deprecation.py`	可用/即将弃用/已弃用三阶段管理
Pipeline 预览	`core/pipeline_preview.py`	排除节点后的清理预览
Pipeline Schema	`core/pipeline_schema.py`	Pipeline Schema 校验
Webhook 回调	`core/webhook_service.py`	事件触发 + HMAC 签名 + 重试
API Gateway	`core/apigw/`	API Token 认证 + 外部触发/状态/模板
执行方案	`models/execution.py` + `scheme_views.py`	预定义节点排除 + 变量覆盖
模板分类	`models/template.py` + `template_category_views.py`	TemplateCategory 可管理分类
模板收藏	`models/template.py` + `template_collect.py`	用户收藏 + 状态追踪
模板 Webhook	`models/webhook.py` + `template_webhook.py`	Webhook 配置 + 投递日志
项目环境变量	`models/env.py` + `project_views.py`	跨模板共享环境变量
外部 API Token	`models/auth.py` + `apigw/auth.py`	ApiToken 认证 + 权限控制
Dry Run	`execution_views.py`	test 原子替换后安全执行
节点执行同步	`execution_views.py`	创建执行时同步 ExecutionNode 行
CMDB Mock	`mock_view/`	8 组 CMDB Mock 数据端点
示例模板自动 Seed	`apps.py`	启动时自动创建示例模板

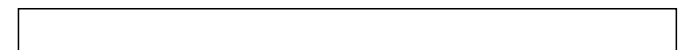
前端

功能	组件	说明
设计画布	`DesignCanvas.vue`	X6 Graph 拖拽式编排
组件面板	`useDesignCanvas.ts`	Stencil 分组拖拽（10 组 37 原子 + 6 网关/事件）
自定义节点	`shapes.ts`	9 种 X6 自定义形状（ops-atom / 网关 / 审批 / 子流程等）
节点属性	`PropertyPanel.vue`	插件选择/版本/参数/超时/重试/变量映射
小地图	`useDesignCanvas.ts`	Minimap 导航
Undo/Redo	`useDesignCanvas.ts`	History 插件 + Ctrl+Z/Y 快捷键

模板选择



`index.vue`

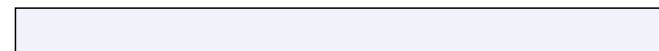


顶部工具栏下拉选择



项目切换器

`ProjectSwitcher.vue`

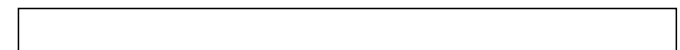


★ 多项目切换 + 角色标识

AI Chat



`index.vue`



浮窗多轮对话

Diff 对比

`DiffModal.vue`

创建向导

``CreateTemplateWizard.vue``

★ 三步向导（输入→生成→保存）

提交向导

`SubmitWizardDialog.vue`

★ 方案选择 + 变量覆盖 + Dry Run

Dry Run 弹窗

`DryRunDialog.vue`

★ Dry Run 结果展示

帮助抽屉

HelpDrawer.vue

★ 快捷键/节点类型/使用指南

实时监控

`MonitorCanvas.vue`

状态管理

`opsflowStore.ts`

执行入口 UI

`DesignCanvas.vue`

画布“运行”按钮 → SubmitWizardDialog

执行记录页面

`ExecutionList.vue` + `ExecutionDetail.vue`

审计日志页面

`opsflow-log/index.vue`

OpsLog 浏览（白卡片 + 风险状态点）

功能	组件	说明
知识库页面	`opsflow-knowledge/index.vue`	OpsKnowledge CRUD（卡片布局）
模板管理页面	`opsflow-template/index.vue`	表格/卡片切换 + 管道可视化 + 细节弹窗 + 调度按钮
调度管理页面	`opsflow-template/schedule.vue` / `ScheduleManager.vue`	调度计划列表 + CRUD + 暂停/恢复
Dashboard	`opsflow-dashboard/index.vue`	4 ECharts + 7 统计卡片 + 用户活动表
审批管理页面	`opsflow-approval/index.vue`	待审批列表 + 通过/拒绝
Webhook 管理	`opsflow-webhook/index.vue`	★ Webhook 配置 + 投递日志
项目管理	`opsflow-project/index.vue`	★ 项目 CRUD + 成员 + 环境变量
统计概览	`opsflow-stats/index.vue`	★ 全局统计概览
插件选择器	`PluginPickerDialog.vue`	按分组树浏览插件
插件可见性	`PluginVisibilityDialog.vue`	★ 项目级插件可见性配置
全局变量面板	`GlobalVariablePanel.vue`	变量列表/编辑/引用计数
变量浏览器	`VariableBrowser.vue`	所有可引用变量查看
项目环境变量	`ProjectEnvVarPanel.vue`	★ 项目级环境变量编辑
执行方案管理	`SchemeManager.vue`	★ 方案 CRUD + 默认方案
执行方案选择	`SchemeSelector.vue`	★ 执行时方案选择
子流程版本徽标	`SubprocessStatusBadge.vue`	版本过期提示
响应式 Undo/Redo	`useDesignCanvas.ts`	history:change 事件同步 Pinia 状态
版本管理弹窗	`VersionDialog.vue`	版本历史展示 + 回滚操作
通用 X6 Graph	`useGraphCanvas.ts`	★ X6 Graph 创建/管理封装
画布校验	`useGraphValidator.ts`	★ 画布节点/连线校验
自动保存	`useAutoSave.ts`	★ 定时自动保存草稿

Pipeline 格式定义

前端画布格式（nodes + edges）

```
{
  "nodes": [
    {
      "id": "node_1",
      "label": "Check Disk",
      "node_type": "",           // 留空或 "atom" = 原子操作
      "atom_type": "disk_check", // 原子类型 (对应 PLUGIN_REGISTRY)
      "params": {"threshold": 80},
      "max_retries": 1,
      "timeout_seconds": 30,
      "risk_level": "low"
    },
    {
      "id": "node_2",
      "label": "Condition?",
      "node_type": "exclusive_gateway" // 网关类型
    }
  ],
  "edges": [
    {"from": "node_1", "to": "node_2", "label": "success"},
    {"from": "node_1", "to": "node_3", "label": "failure"}
  ]
}
```

节点类型

node_type	含义	出度	入度
(空) / "atom"	原子操作	1-2	≥1
"exclusive_gateway"	排他网关	≥1	≥1
"parallel_gateway"	并行网关	≥1	≥1
"conditional_parallel_gateway"	条件并行网关	≥1	≥1
"converge_gateway"	汇聚网关	1	≥2
"approval"	审批节点	1	≥1
"subprocess"	子流程节点	1	≥1
"start_event"	流程起点	1	0
"end_event"	流程终点	0	≥1

出边标签规则

- 原子节点出度=2 时，标签必须为 [success] 和 [failure]
- 排他网关多出边时，标签区分条件分支
- 并行网关出边标签不参与条件判断（全执行）

执行状态机

